

Arch Technologies
Machine Learning / Data Science Internship

Task 1: Email Spam Classification

Full Name	Ali Rashid
Internship Domain	Machine Learning / Data Science
Email Address	ar2701699@gmail.com
Phone Number	0305-7357819

1. Objective

The objective of this task is to build a machine learning model that classifies text messages as spam or not spam (ham) using a labelled dataset. The workflow covers text pre-processing, feature extraction, model training with multiple algorithms, and evaluation of each model's performance.

2. Dataset

A labelled SMS spam collection dataset was used, consisting of 5,572 real text messages tagged as either “ham” (legitimate) or “spam”. The class distribution is as follows:

- Ham (not spam): 4,825 messages (86.6%)
- Spam: 747 messages (13.4%)

This is the same style of labelled, real-world text dataset referenced in the task brief, and is well suited to a spam/ham binary classification problem.

3. Methodology

3.1 Text Pre-processing

- Converted all text to lowercase
- Removed URLs and numeric tokens
- Stripped punctuation and normalised whitespace
- Removed English stopwords during vectorisation

3.2 Feature Extraction

Cleaned text was converted into numerical features using TF-IDF (Term Frequency – Inverse Document Frequency) vectorisation, using both unigrams and bigrams and a vocabulary capped at 3,000 features. TF-IDF weighs words by how distinctive they are to a message, which helps the model pick up on spam-indicative phrasing (e.g. “free entry”, “claim now”) rather than just common words.

3.3 Model Training

The dataset was split 80/20 into training and test sets (stratified by class). Three classification algorithms were trained and compared:

- Multinomial Naive Bayes
- Logistic Regression
- Linear Support Vector Machine (SVM)

3.4 Evaluation

Each model was evaluated on the held-out test set (1,115 messages) using accuracy, precision, recall, and F1-score, since accuracy alone can be misleading on an imbalanced dataset like this one.

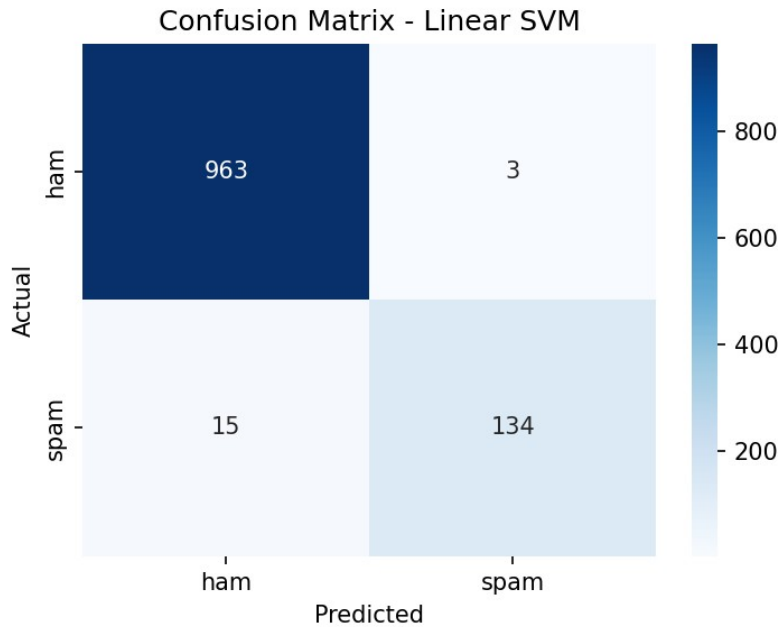
4. Results

Model	Accuracy	Precision	Recall	F1-score
Multinomial Naive Bayes	97.2%	100%	79.2%	0.884
Logistic Regression	97.0%	100%	77.2%	0.871
Linear SVM (best)	98.4%	97.8%	89.9%	0.937

The Linear SVM model achieved the best overall performance, with the highest accuracy (98.4%) and F1-score (0.937), correctly catching 134 of 149 spam messages in the test set while misclassifying only 3 legitimate messages as spam.

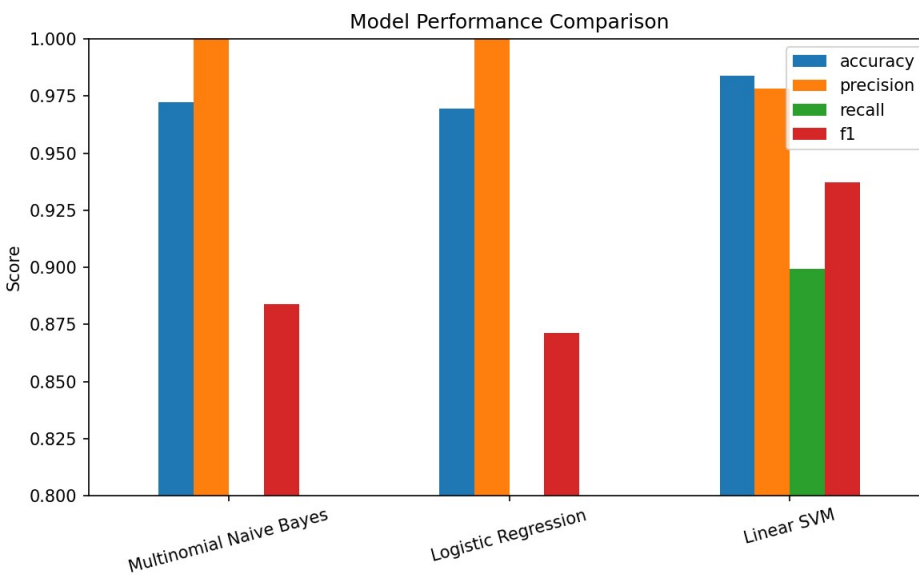
5. Screenshots

5.1 Confusion Matrix (Best Model – Linear SVM)



Out of 1,115 test messages, 963 ham and 134 spam messages were classified correctly, with only 3 ham messages misclassified as spam and 15 spam messages missed.

5.2 Model Performance Comparison



6. Conclusion

A complete spam classification pipeline was built and evaluated end-to-end: data cleaning, TF-IDF feature extraction, training of three classifiers, and comparison of their performance. The Linear SVM model was selected as the best performer and is recommended as the final model for this task, achieving 98.4% accuracy and a 0.937 F1-score on unseen data.

7. Code Section

The full Python implementation is included below.

```
"""
Task 1: Email Spam Classification
Arch Technologies - ML/Data Science Internship

Pipeline: load data -> clean/preprocess text -> TF-IDF features ->
train multiple models -> evaluate -> save results & plots.
"""

import re
import string
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)

RANDOM_STATE = 42

# -----
# 1. Load dataset
# -----
df = pd.read_csv("sms.tsv", sep="\t", header=None, names=["label", "text"])
print("Dataset shape:", df.shape)
print(df["label"].value_counts())

# -----
# 2. Preprocessing
# -----
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " ", text)      # urls
    text = re.sub(r"\d+", " ", text)                # numbers
    text = text.translate(str.maketrans("", "", string.punctuation))
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["clean_text"] = df["text"].apply(clean_text)
df["label_num"] = df["label"].map({"ham": 0, "spam": 1})

# -----
# 3. Train/test split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    df["clean_text"], df["label_num"],
    test_size=0.2, random_state=RANDOM_STATE, stratify=df["label_num"]
)
```

```

# -----
# 4. Feature extraction (TF-IDF)
# -----
vectorizer = TfidfVectorizer(stop_words="english", max_features=3000, ngram_range=(1, 2))
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# -----
# 5. Train models
# -----
models = {
    "Multinomial Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Linear SVM": LinearSVC(),
}

results = {}
for name, model in models.items():
    model.fit(X_train_tfidf, y_train)
    preds = model.predict(X_test_tfidf)
    results[name] = {
        "accuracy": accuracy_score(y_test, preds),
        "precision": precision_score(y_test, preds),
        "recall": recall_score(y_test, preds),
        "f1": f1_score(y_test, preds),
        "preds": preds,
    }
    print(f"\n=== {name} ===")
    print(classification_report(y_test, preds, target_names=["ham", "spam"]))

# -----
# 6. Pick best model (highest F1) and plot confusion matrix
# -----
best_name = max(results, key=lambda k: results[k]["f1"])
best_preds = results[best_name]["preds"]
print(f"\nBest model: {best_name}")

cm = confusion_matrix(y_test, best_preds)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["ham", "spam"], yticklabels=["ham", "spam"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"Confusion Matrix - {best_name}")
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150)
plt.close()

# -----
# 7. Summary comparison chart
# -----
summary_df = pd.DataFrame({k: {m: v[m] for m in ["accuracy", "precision", "recall", "f1"]}
                           for k, v in results.items()}).T
summary_df.to_csv("model_comparison.csv")
print("\nModel comparison:\n", summary_df)

summary_df.plot(kind="bar", figsize=(8, 5), ylim=(0.8, 1.0))
plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig("model_comparison.png", dpi=150)
plt.close()

print("\nDone. Saved: confusion_matrix.png, model_comparison.png, model_comparison.csv")

```